

task_bridge

सामग्री

आपका टास्क बोर्ड और आपका सत्य-स्रोत, दोनों दिशाओं में बेदाग तालमेल में।

सारांश

task_bridge, Go में एक सामान्य, डिक्पल्ड, द्विदिशात्मक टास्क/बोर्ड सिंक इंजन है। यह किसी प्रोजेक्ट के कार्य-आइटम्स SQLite सत्य-स्रोत को उसके ट्रैकर दस्तावेजों और रिमोट बोर्ड (प्रथम लक्ष्य: ClickUp; Jira/Linear नियोजित) के साथ डिटर्मिनिस्टिक लास्ट-एडिट-विन्स, ड्राई-रन-फर्स्ट, और कभी-भ्रष्ट-नहीं होने वाली प्रक्रिया के माध्यम से तालमेल में रखता है।

संक्षिप्त विवरण

एक प्रोजेक्ट-अज्ञेयवादी Go सबमॉड्यूल जो द्विदिशात्मक रूप से SQLite कार्य-आइटम्स सत्य-स्रोत ↔ ट्रैकर दस्तावेज ↔ रिमोट बोर्ड (प्रथम: ClickUp) को सिंक करता है। डिटर्मिनिस्टिक लास्ट-एडिट-विन्स, ड्राई-रन-फर्स्ट, HMAC-सत्यापित वेबहुक्स; हर क्रेडेंशियल और आईडी उपभोक्ता द्वारा रनटाइम पर इंजेक्ट की जाती है।

विस्तृत विवरण

आखिरकार हर टीम एक ही काम के दो अलग-अलग रिकॉर्ड रखती है: असली — कोड, दस्तावेज, आंतरिक डेटाबेस — और दूसरा, जिसे मैनेजर देखते हैं, जैसे ClickUp का बोर्ड। दोनों में तालमेल टूट जाता है जैसे ही किसी एक पक्ष में बदलाव होता है, और उन्हें मैनुअल रूप से मिलाना वही उबाऊ, त्रुटि-प्रवण काम है जिसे कोई भरोसेमंद तरीके से नहीं करता। task_bridge इसी अंतर को मिटाने के लिए बनाया गया है, जिसमें तीनों प्रतिनिधित्वों को एक ही प्रणाली के रूप में देखा जाता है जिसे तालमेल में रखा जाना चाहिए: प्रोजेक्ट का कार्य-आइटम्स SQLite एकल-सत्य-स्रोत, उसके ट्रैकर दस्तावेज, और एक रिमोट बोर्ड — पहला समर्थित बोर्ड ClickUp है, जबकि Jira और Linear भविष्य में जोड़े जाने की योजना है। सिंक्रोनाइजेशन डिटर्मिनिस्टिक (लास्ट-एडिट-विन्स), ड्राई-रन-फर्स्ट है, और इसे एक अटल वादे के इर्द-गिर्द डिज़ाइन किया गया है: यह कभी डेटा को भ्रष्ट या खोएगा नहीं, और कभी चुपचाप किसी एक पक्ष को पुराना नहीं छोड़ेगा। इस क्षेत्र में एक लापरवाह सिंक एक हफ्ते के काम को मिटा सकता है, इसलिए इस सुरक्षा-प्रक्रिया का यही उद्देश्य है।

वास्तुशिल्पीय रूप से यह एक सख्त सबमॉड्यूल है जिसे अन्य प्रोजेक्ट्स द्वारा उपयोग किया जाता है और यह पूरी तरह प्रोजेक्ट-अज्ञेयवादी है, जैसा कि संविधान के डिकपलिंग अनुबंध (§11.4.28) में निर्धारित है: यह कोई प्रोजेक्ट-विशिष्ट मान नहीं भेजता, और हर क्रेडेंशियल, बोर्ड/फ़ोल्डर आईडी, आइटम-की फ़ील्ड, और डेटाबेस पथ उपभोक्ता द्वारा रनटाइम पर pkg/config.Config के माध्यम से इंजेक्ट किया जाता है। मॉड्यूल की परतें स्पष्ट हैं: एक CLI (reconcile/push/pull/resolve/status/conflicts/init) और एक दीर्घकालिक डेमन (वेबहुक रिसीवर + क्रॉन रीकंसील); MIT-लाइसेंस प्राप्त raksul/go-clickup पर आधारित एक पतला क्लाइंट रैपर; एक रिज़ॉल्वर जो बोर्ड/फ़ोल्डर URL को लाइव API प्रोक्स के माध्यम से आईडी में बदलता है (कोई URL-ग्रामर अनुमान नहीं); स्थानीय कार्य-आइटम्स और रिमोट टास्क फ़ील्ड्स के बीच एक मैपर; एक लास्ट-एडिट-विन्स सिंक इंजन जिसमें स्पष्ट विरोध समाधान होते हैं; और एक वेबहुक रिसीवर जो X-Signature HMAC-SHA256 सत्यापित करता है। यह अपनी परिपक्वता के बारे में ईमानदार है: यह P1 ढाँचा है — लेआउट, इंटरफ़ेस, एंटीपाइंड्स, और डिकपलिंग सीमा तैयार हैं, लेकिन सिंक लॉजिक और लाइव ClickUp कॉल अभी लागू नहीं किए गए हैं (हर स्टब एक स्पष्ट नॉट-इम्प्लीमेंटेड एरर लौटाता है, नो-फेक्स नियम के अनुसार)।

हमने इसे क्यों बनाया

टीमें काम की “असली” स्थिति कोड/दस्तावेज़ों में रखती हैं जबकि मैनेजर ClickUp जैसे बोर्ड पर निर्भर रहते हैं — और दोनों लगातार अलग होते जाते हैं। task_bridge इन्हें एक प्रणाली में बदल देता है, जो डिटर्मिनिस्टिक और सुरक्षित तरीके से सिंक करता है ताकि कोई भी पक्ष पुराना या गलत न हो जाए।

सामग्री

यह क्यों एक गेम-चेंजर है

दो-तरफ़ा बोर्ड सिंक आमतौर पर एक बार की, हार्ड-वायर्ड इंटीग्रेशन होती है जिसे हर टीम खराब तरीके से दोबारा बनाती है। task_bridge इसे एक पुनःप्रयोगी, क्रेडेंशियल-इंजेक्टेड लाइब्रेरी के रूप में पुनः परिभाषित करता है जिसमें सख्त डेटा-सुरक्षा गारंटी शामिल है — पहले ड्राई-रन, फिर नियतात्मक अंतिम-संपादन-विजेता, HMAC-सत्यापित इवेंट्स — ताकि कोई भी प्रोजेक्ट भरोसेमंद बोर्ड इंटीग्रेशन को कॉन्फ़िगरेशन इंजेक्ट करके अपना सके, बजाय इसके कि हर बार एक नया नाज़ुक कनेक्टर लिखा जाए जो उसके आंतरिक तंत्र से जुड़ा हो।

क्या है नवीन

- तीन-तरफ़ा द्विदिश सिंक: SQLite SSoT ↔ ट्रैकर दस्तावेज़ ↔ रिमोट बोर्ड।
- पूर्ण डिकपलिंग (§11.4.28): प्रोजेक्ट के कोई मूल्य नहीं; सब कुछ रनटाइम पर इंजेक्ट किया जाता है।
- लाइव-API URL→ID रिज़ॉल्यूशन, नाज़ुक URL-ग्रामर पार्सिंग के बजाय।
- लाइव इवेंट्स के लिए HMAC-SHA256-सत्यापित वेबहुक इनजेक्शन।

चुनौतियाँ और समाधान

- तीन स्रोतों में डेटा सुरक्षा: नियतात्मक अंतिम-संपादन-विजेता, पहले ड्राई-रन, और स्पष्ट विरोधाभासी परिणामों से हल।
- कपलिंग के बिना पुनःप्रयोगिता: pkg/config इंजेक्शन बाउंड्री के ज़रिए हल (प्रोजेक्ट-विशिष्ट कुछ भी शिप नहीं किया जाता)।
- विश्वसनीय बोर्ड पहचान: लाइव API प्रोब्स के ज़रिए URL को ID में बदलकर हल।
- ईमानदार ढाँचा: अनइम्प्लीमेंटेड स्टब्स को स्पष्ट “नोट-इम्प्लीमेंटेड” एरर लौटाकर हल (कोई नकली समाधान नहीं)।

तकनीकी स्टैक (क्यों + कैसे)

- **Go** — इंजन, CLI (cmd/task_bridge), और डेमन (cmd/task_bridged)।
- **SQLite** — कार्य योग्य आइटम्स का एकल-स्रोत-सत्य।
- **raksul/go-clickup (MIT)** — ClickUp ट्रांसपोर्ट रैपर।
- **HMAC-SHA256** — वेबहुक हस्ताक्षर सत्यापन।
- **cron + वेबहुक्स** — डेमन रीकन्साइल + लाइव-इवेंट इनजेक्शन।
- **pkg/config** — रनटाइम क्रेडेंशियल/ID इंजेक्शन बाउंड्री।

स्थिति की ईमानदारी: यह एक **P1** ढाँचा है — सिंक लॉजिक अभी लागू नहीं किया गया है। इसे शिफ्ट के रूप में प्रस्तुत न करें।